RCNN

RCNN

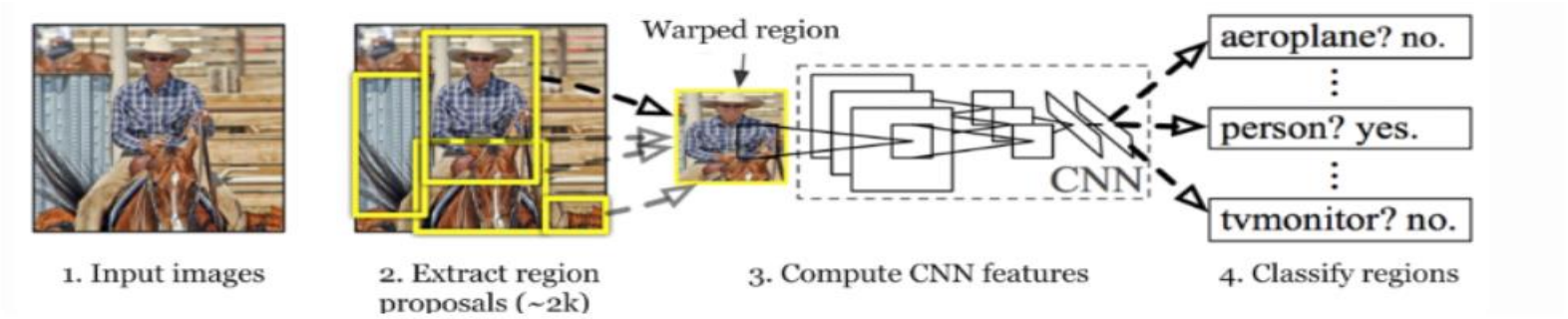
Developed by Ross Girshick et al. (2014).
RCNN stands for Region-based Convolutional Neural Network.
A two-stage object detection method:

- First, generate region proposals using selective search.
- Then, apply CNN on each region for classification and bounding box regression.

- Drawback:

Time-consuming due to multiple CNN evaluations.

RCNN Architecture



R-CNN architecture

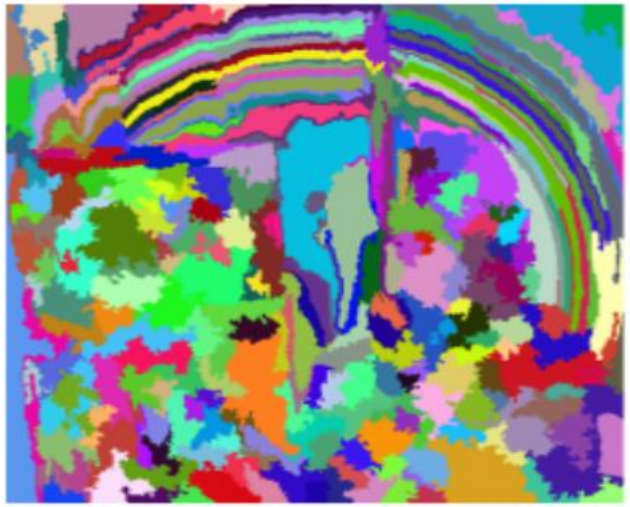
1.Region Proposal: Selective search algorithm generates candidate region proposals.
2.Feature Extraction: Apply CNN on each region proposal.
3.Classification: Classify each region using a pre-trained SVM (Support Vector Machine).
4.Bounding Box Regression: Adjust bounding boxes using regression models.

Limitations:

- ✓ Slow due to redundant CNN computations.
- ✓ Requires saving features and training separately for SVMs and bounding box regressors.

Selective Search Algorithm

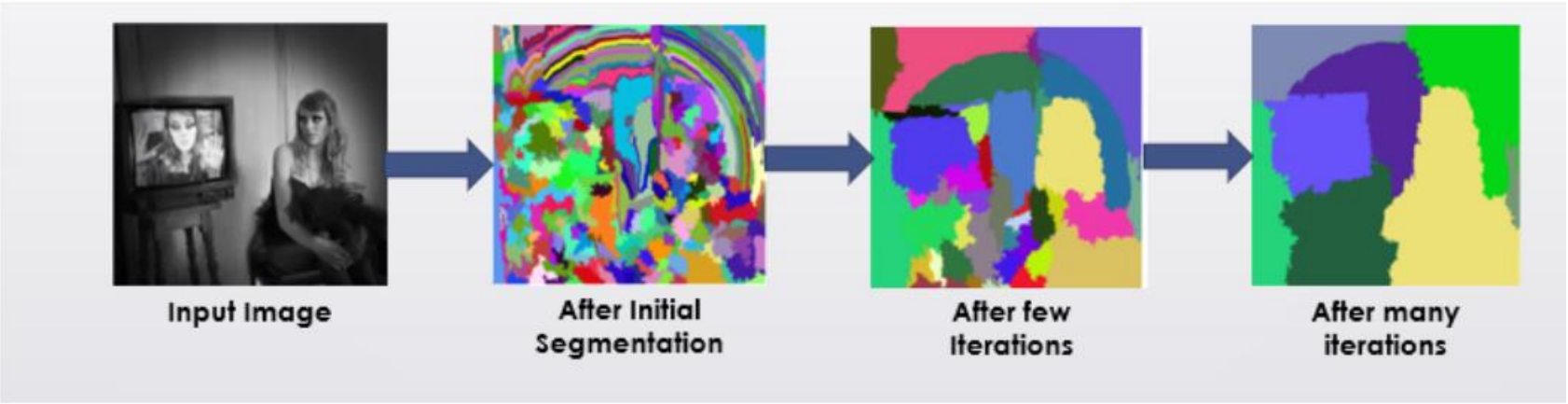
Generate initial sub-segmentation of input image using the method describe by *Felzenszwalb et al* in his paper “Efficient Graph-Based Image Segmentation “.



Selective Search Algorithm

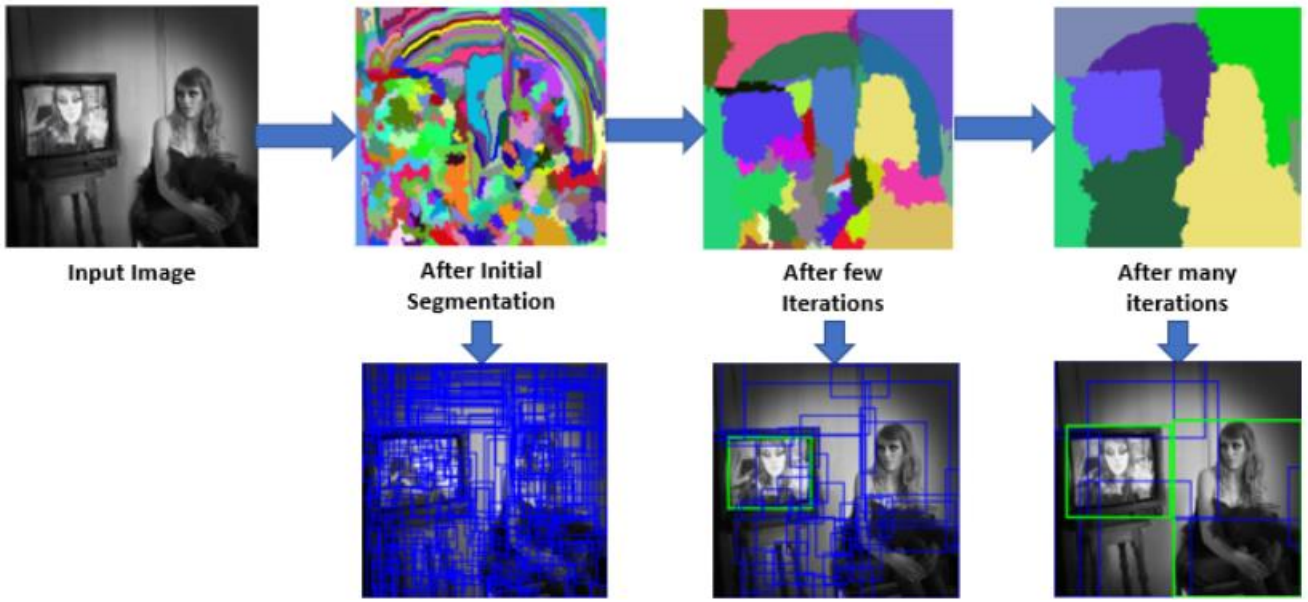
Recursively combine the smaller similar regions into larger ones. We use Greedy algorithm to combine similar regions to make larger regions. The algorithm is written below.

```
Greedy Algorithm :
1. From set of regions, choose two that are most similar.
2. Combine them into a single, larger region.
3. Repeat the above steps for multiple iterations.
```



Selective Search Algorithm

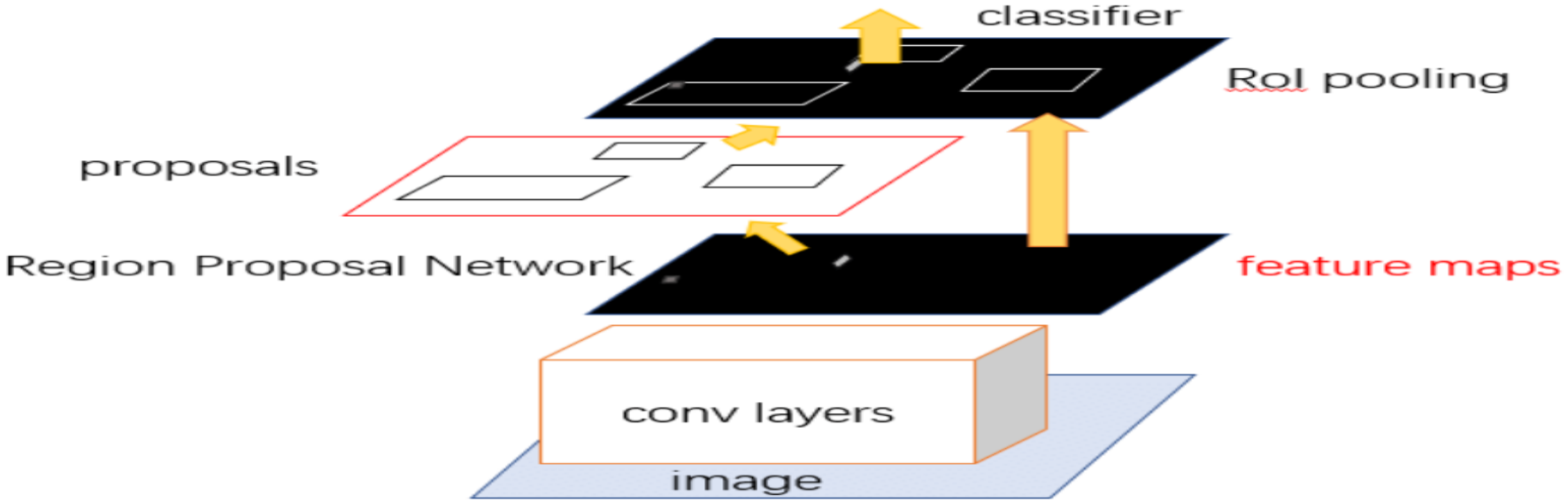
Use the segmented region proposals to generate candidate object locations.



Faster RCNN

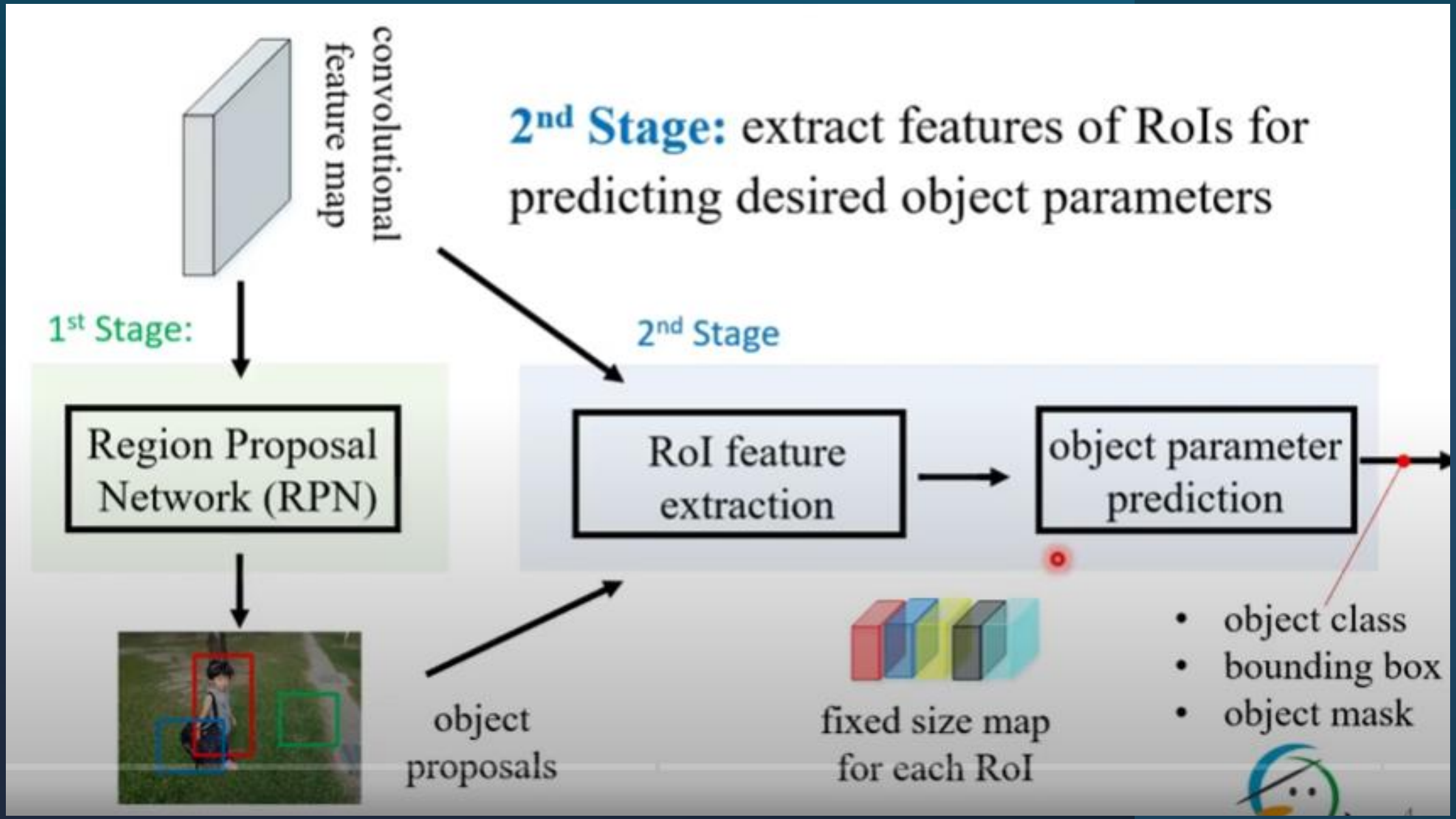
✓ Faster RCNN is an advancement over RCNN and Fast RCNN.
✓ Introduced by Shaoqing Ren et al. (2015).
✓ **Key Improvement:** Uses a Region Proposal Network (RPN) to generate region proposals.
Significantly faster than RCNN and Fast RCNN by integrating region proposal generation with object detection.

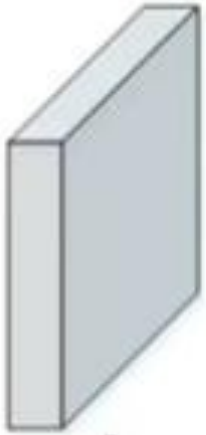
Faster RCNN Architecture



Convolutional Layers: Feature extraction from the input image using shared convolutional layers.
Region Proposal Network (RPN): Proposes potential regions based on features.
RoI Pooling: Converts regions of different sizes into fixed-size feature maps.
Fully Connected Layers: Classifies the objects and refines bounding boxes.

Faster RCNN combines region proposal generation and classification into one network, reducing time and complexity


convolutional feature map
2nd Stage: extract features of RoIs for predicting desired object parameters
1st Stage:
2nd Stage
Region Proposal Network (RPN)
RoI feature extraction
object parameter prediction
object proposals
fixed size map for each RoI
object class
bounding box
object mask



convolutional feature map

2nd Stage: extract features of RoIs for predicting desired object parameters

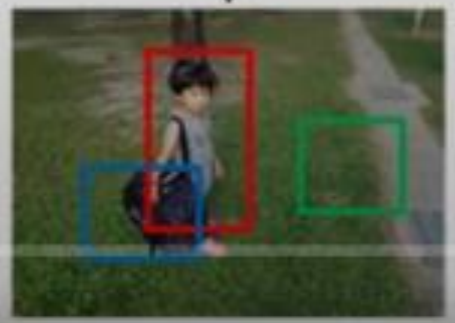
1st Stage:

2nd Stage

Region Proposal Network (RPN)

- RoI Pooling
- RoI Align

object parameter prediction

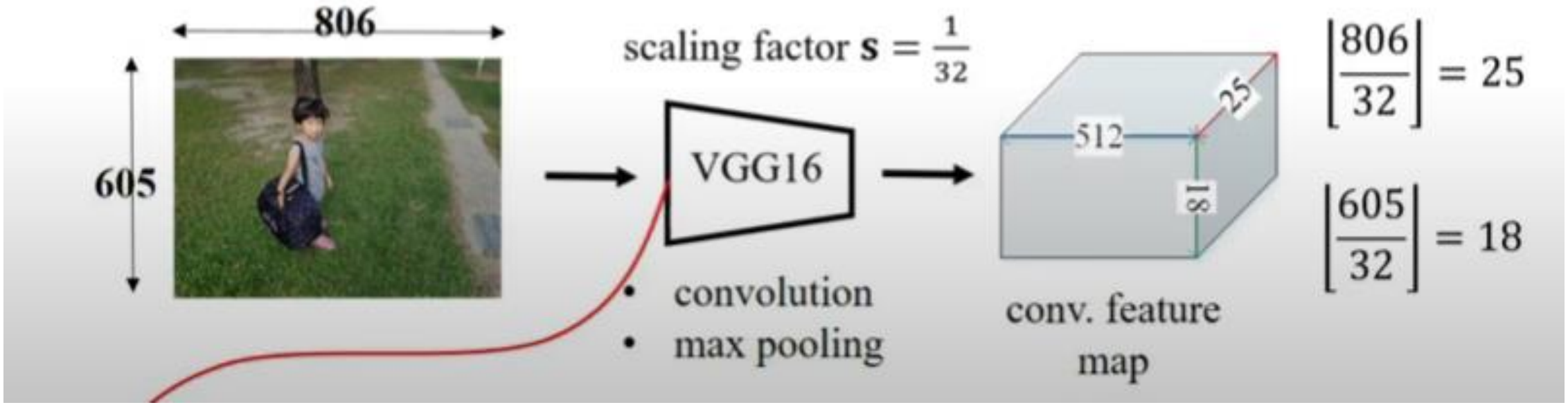


object proposals

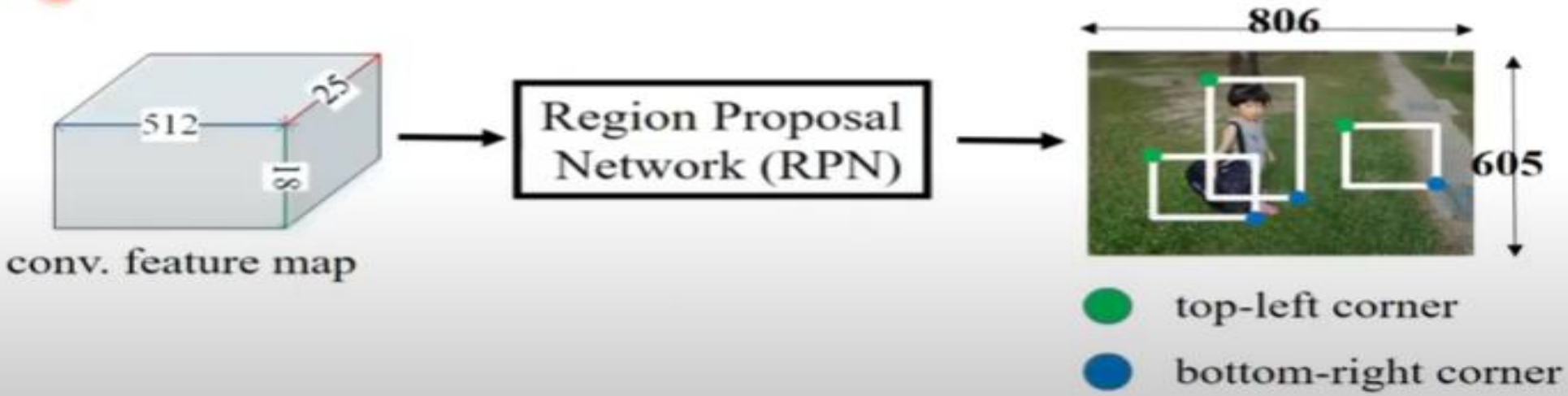
fixed size map for each RoI

- object class
- bounding box
- object mask

- RoI Feature Extraction: input
  - Feature Map: feature of the input image obtained from a deep convolutional neural network.



- RoI Feature Extraction: input
  - RoI List: object proposals from RPN

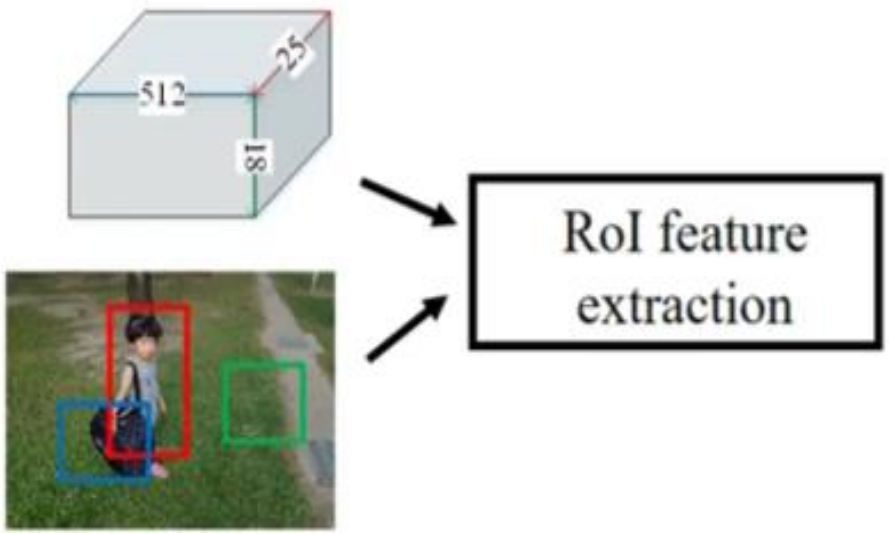


1st RoI: (224, 89), (439, 426)

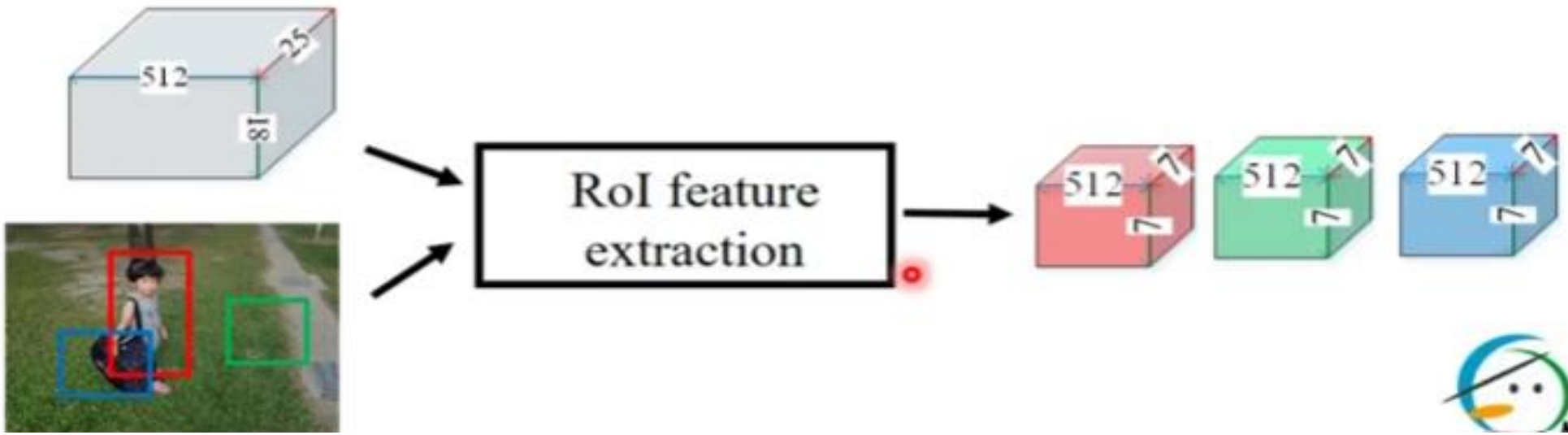
2nd RoI: (537, 217), (732, 383)

3rd RoI: (134, 312), (355, 489)

- RoI Feature Extraction: output
  - RoI feature maps: each one is the feature map within a RoI and with the fixed size $k \times k \times c$
    - k: pre-defined size ($k = 7$ in original paper)
    - c: num. of channels of input conv. feature map



- RoI Feature Extraction: output
  - RoI feature maps: each one is the feature map within a RoI and with the fixed size $k \times k \times c$
    - k: pre-defined size ($k = 7$ in original paper)
    - c: num. of channels of input conv. feature map



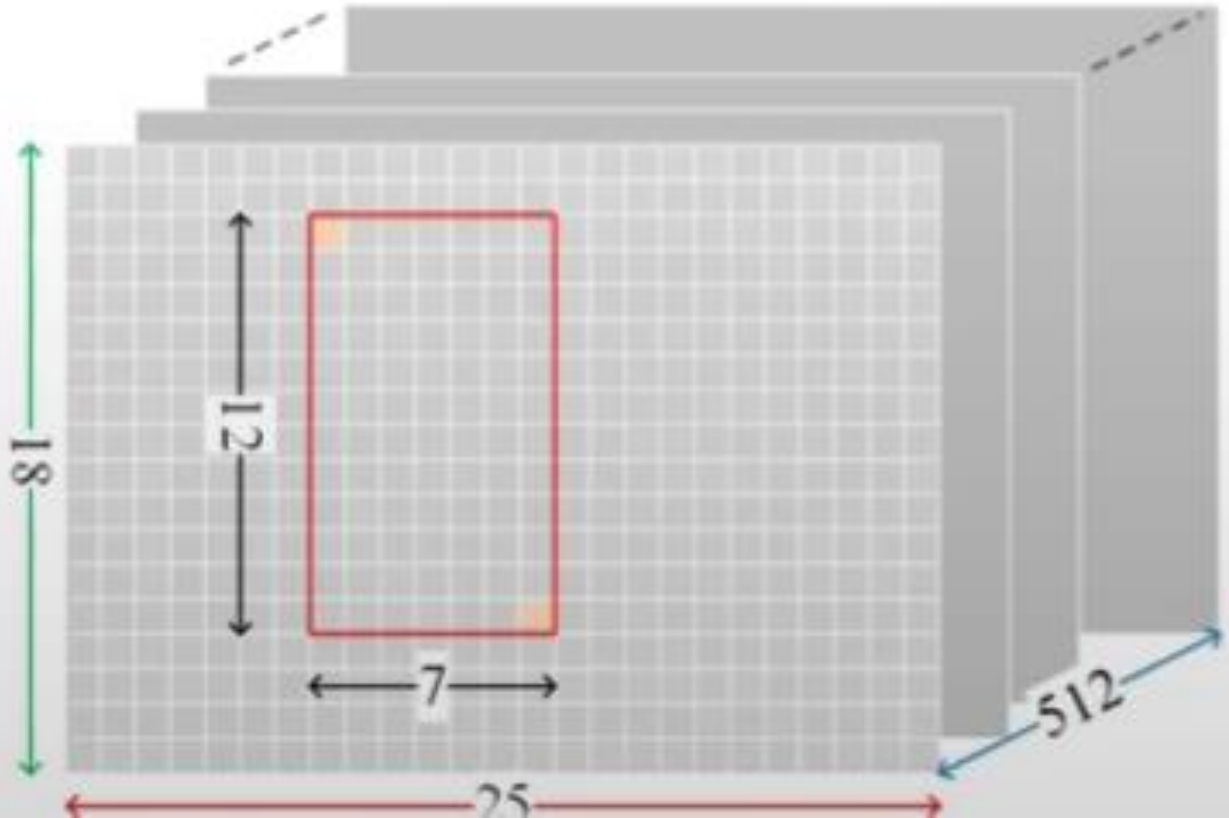
- Overview
  - RoI pooling is firstly proposed in Fast R-CNN for object detection.

    Ross Girshick, "Fast R-CNN," *IEEE ICCV*, 2015

  - RoI pooling uses max pooling operation for computing the RoI feature map.
    - make the conv. feature map be reused for all RoIs
    - make the architecture be trained end-to-end.

- Overview: three steps
  - Step 1 (RoI mapping): map RoI bounding box from image to conv. feature map by quantization
  - Step 2 (RoI division): divide the mapped RoI into $k \times k$ grids by quantization
  - Step 3 (max pooling): find the maximum of all points in a grid

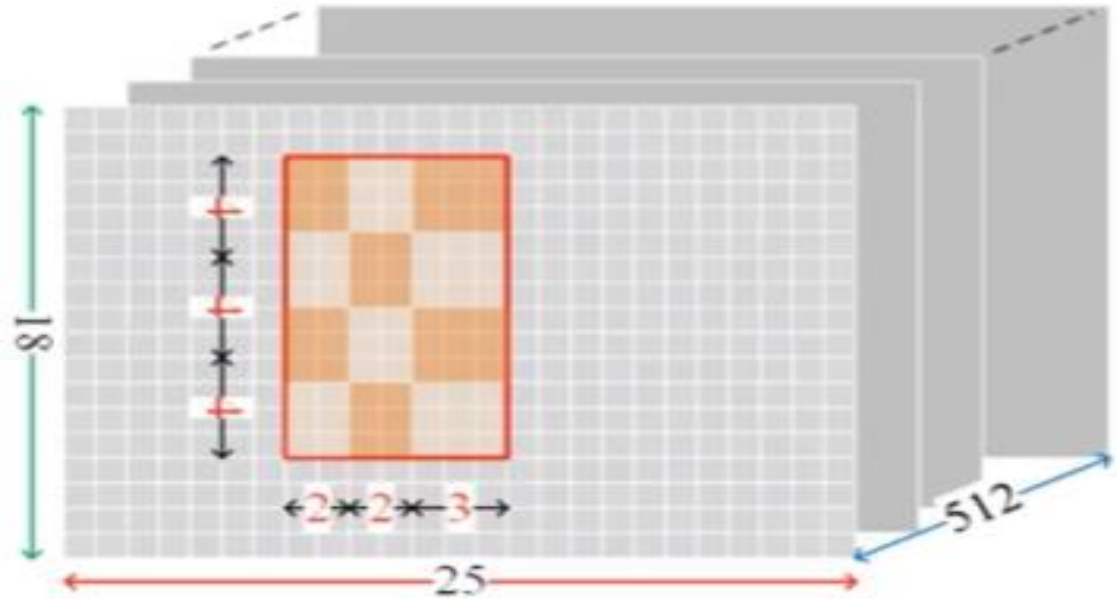
- Pooling Steps: RoI division
  - divide the width/height of the mapped RoI by k
  - quantize the width/height by floor operator



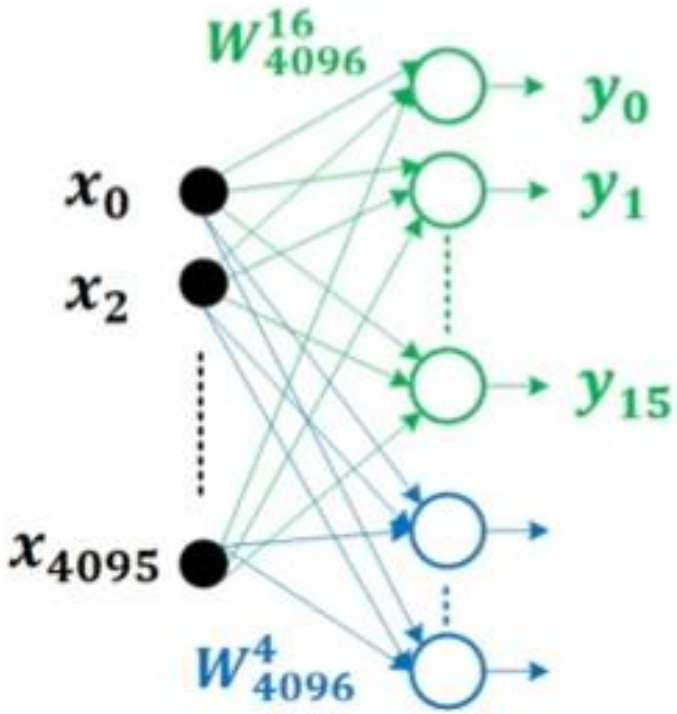
grid width: $\left\lfloor \frac{7}{3} \right\rfloor = \lfloor 2.33 \rfloor = \mathbf{2}$

grid height: $\left\lfloor \frac{12}{3} \right\rfloor = \lfloor 4.00 \rfloor = \mathbf{4}$

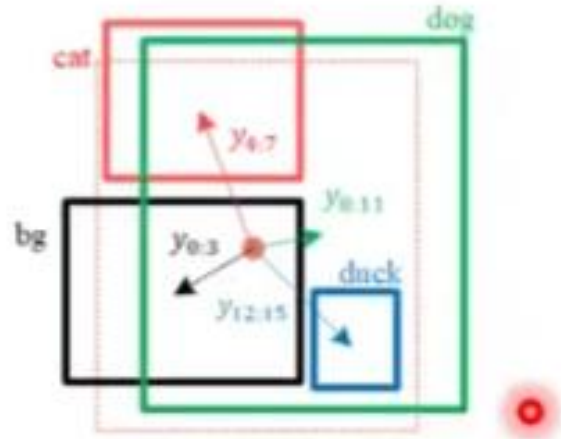
- Pooling Steps: max pooling
  - take the maximum of points in each grid
  - aggregate the results in all channels



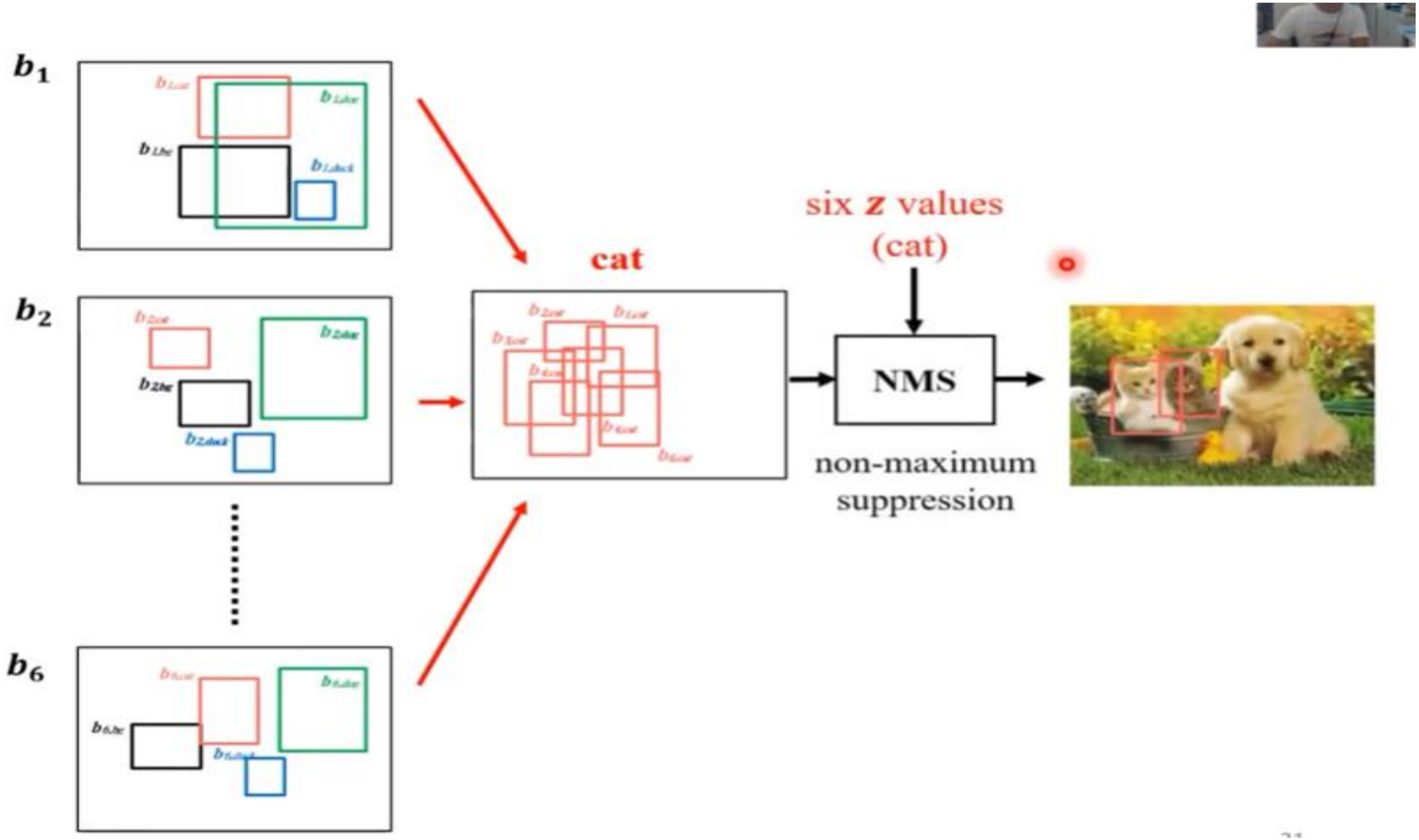
- Detection Network
  - apply two fully connected branches for
    - regression: $4096 \rightarrow (N_{class} + 1) \times 4$ background (bg)
    - classification: $4096 \rightarrow (N_{class} + 1)$ offset: (d_x, d_y, d_w, d_h)



$\hat{C} = \{\mathbf{bg}, \mathbf{cat}, \mathbf{dog}, \mathbf{duck}\}$



6 object proposals


b_1
b_2
b_6
cat
six z values
(cat)
NMS
non-maximum
suppression

(1) Convolution layer: It can extract the feature map of the entire picture.

(2) Region Proposal Network (RPN): As the core part of Faster R-CNN, RPN supersedes Selective Search method in Fast R-CNN algorithm to generate a region proposal. Using RPN to obtain a region proposal can more quickly and efficiently use the CNN network. RPN generates anchors while generating a region proposal. The judgement function determines whether the anchors are foreground or background, and then adjusts the anchors through border regression to obtain an accurate region proposal.

(3) RoI pooling: It can deal with the problem that different sizes of feature maps input to the network with fully connected layer. The fixed size is obtained by up-sampling.

(4) Classification layer and regression layer:

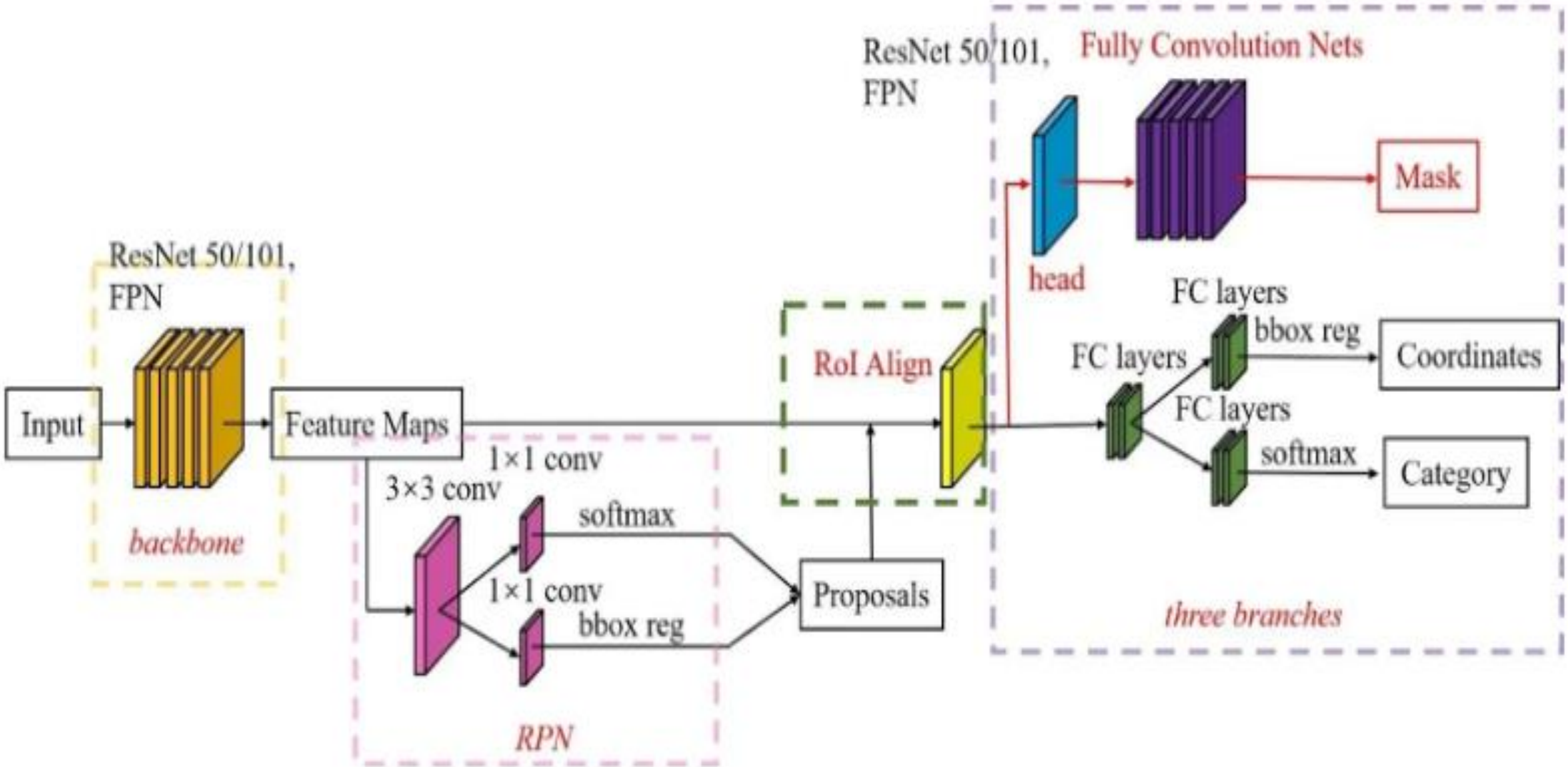
- The classification layer is responsible for judging which class an object belongs to;
- The regression layer fine adjusts the positions of regions of interest (RoIs) to obtain the final object detection result.

Masked Rcnn

- Mask RCNN extends Faster RCNN by adding a branch for pixel-level segmentation.
- Developed by He et al. (2017).
- Simultaneously performs:
- **Object Detection**: Bounding box generation.
- **Instance Segmentation**: Pixel-wise mask generation for each detected object.
- Applied in tasks like autonomous driving, medical image analysis, and video analysis.

Masked Rcnn Architecture

1. Mask RCNN adds an additional mask prediction branch to Faster RCNN:
2. **Backbone Network**: Feature extraction (ResNet or other CNNs).
3. **RPN**: Generates region proposals.
4. **RoIAlign**: Aligns RoI feature maps for better accuracy (improves over RoIPool by handling fractional pixel locations).
5. **Classification and Bounding Box Regression**.
6. **Mask Head**: Predicts a binary mask for each region, indicating the object pixels.

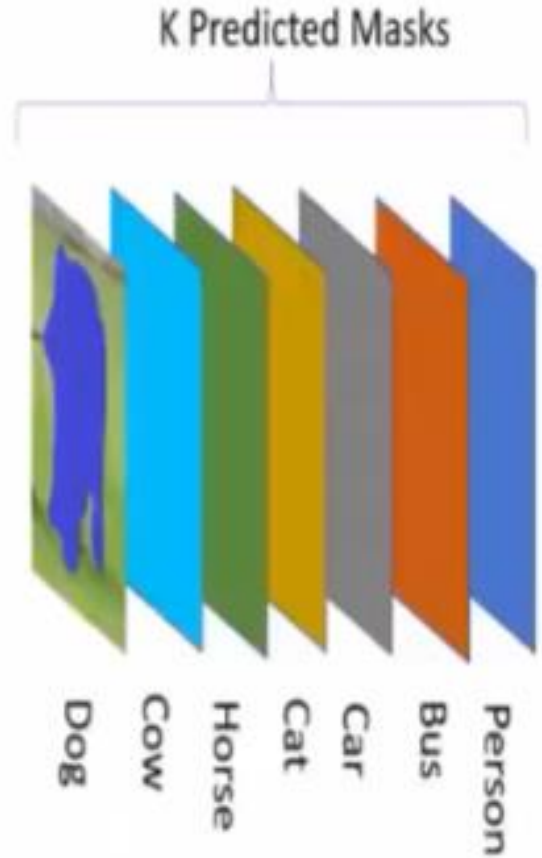


The Mask R-CNN framework for instance segmentation. Source

Mask head

- responsible for generating segmentation masks for each region proposal.
- The head uses the aligned features obtained through ROIAlign to predict a binary mask for each object, delineating the pixel-wise boundaries of the instances.
- The Mask Head is typically composed of several convolutional layers followed by upsample layers (deconvolution or transposed convolution layers).

Mask Loss

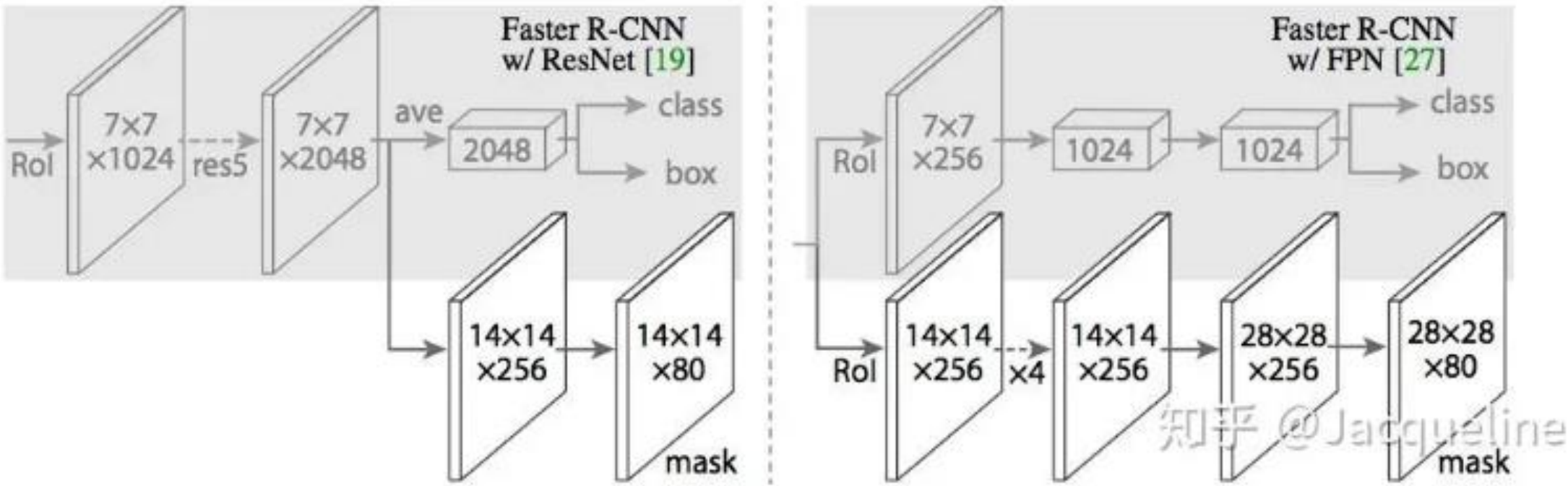


Head – Mask Prediction

- Fully convolutional
- K · (m × m) sigmoid outputs
  - pixel-wise binary classification
  - one mask for each class
- L_{mask}: mean binary cross-entropy

$$L_{box} = L_{cls} + L_{reg}$$

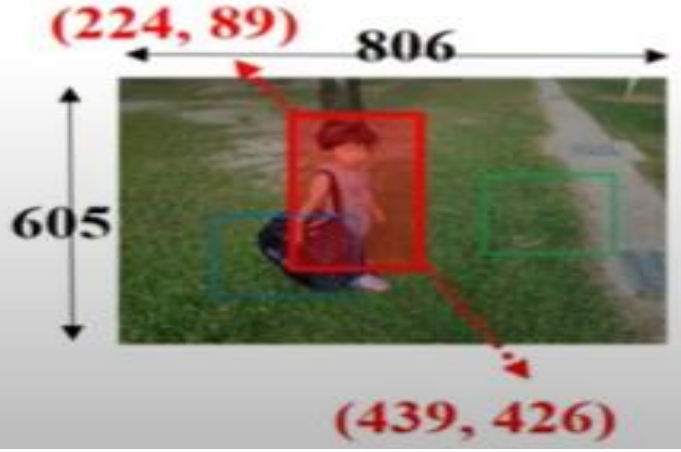
$$L_{final} = L_{box} + L_{mask}$$



Mask Head Structure. Source

RoiAlign

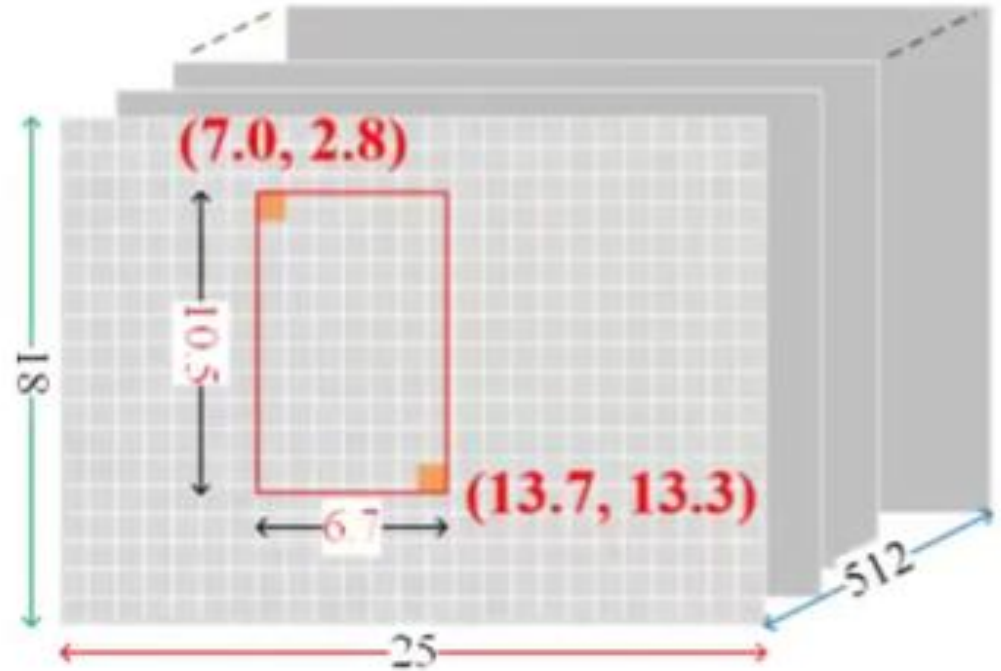
- Align Steps
  - RoI mapping: multiple the scaling factor
  - RoI division: divide width/height by k



ROI Align

- Overview: four steps
  - Step 1 (RoI mapping): multiply the scaling factor to map RoI to conv. feature map
  - Step 2 (RoI division): divide the width/height of mapped RoI by k to have $k \times k$ grids.
  - Step 3 (Interpolation): interpolate the values of all sampling points (each grid has $s \times s$ points)
  - Step 4 (max pooling): find the maximum of all $s \times s$ sampling points in a grid.

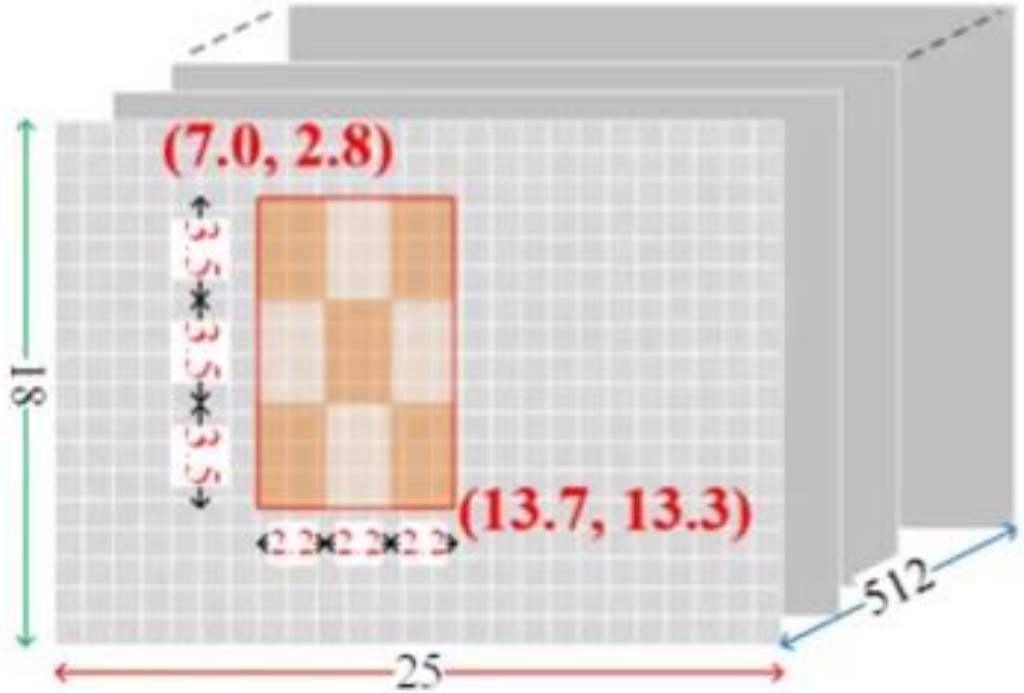
- Align Steps
  - RoI mapping: multiple the scaling factor
  - RoI division: divide width/height by k



$$\left(\left\lfloor 224 \times \frac{1}{32} \right\rfloor, \left\lfloor 89 \times \frac{1}{32} \right\rfloor\right) = (7.0, 2.8)$$

$$\left(\left\lfloor 439 \times \frac{1}{32} \right\rfloor, \left\lfloor 426 \times \frac{1}{32} \right\rfloor\right) = (13.7, 13.3)$$

- Align Steps
  - RoI mapping: multiple the scaling factor
  - RoI division: divide width/height by k

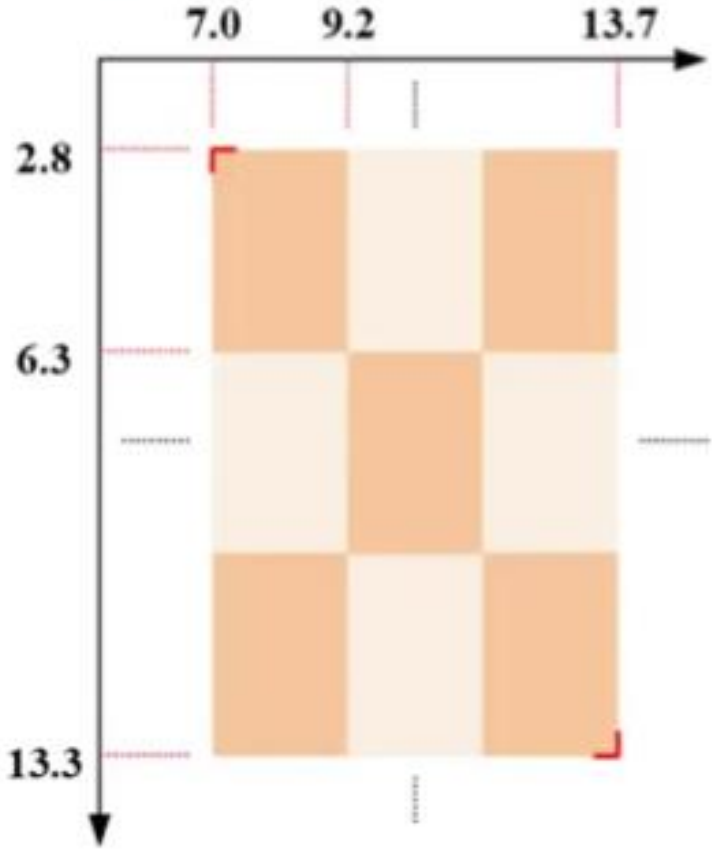


$(k = 3)$

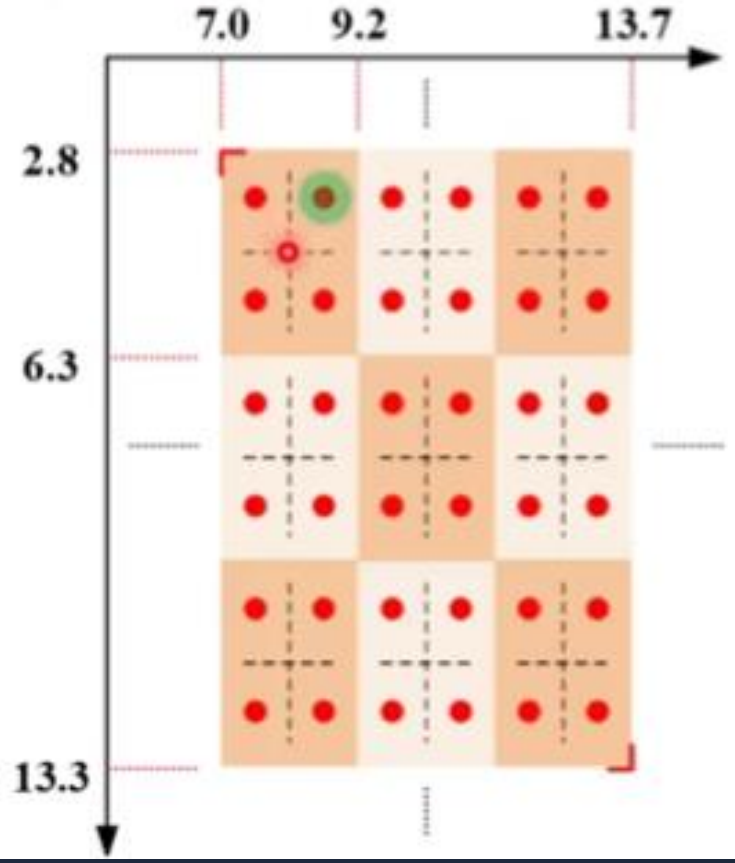
grid width: $6.7 \div 3 = 2.2$

grid height: $10.5 \div 3 = 3.5$

- Align Steps: interpolation
  - divide each grid into $s \times s$ cells ($s = 2$)
  - take the centroids of cells as sampling points



- Align Steps: interpolation
  - divide each grid into $s \times s$ cells $(s = 2)$
  - take the centroids of cells as sampling points



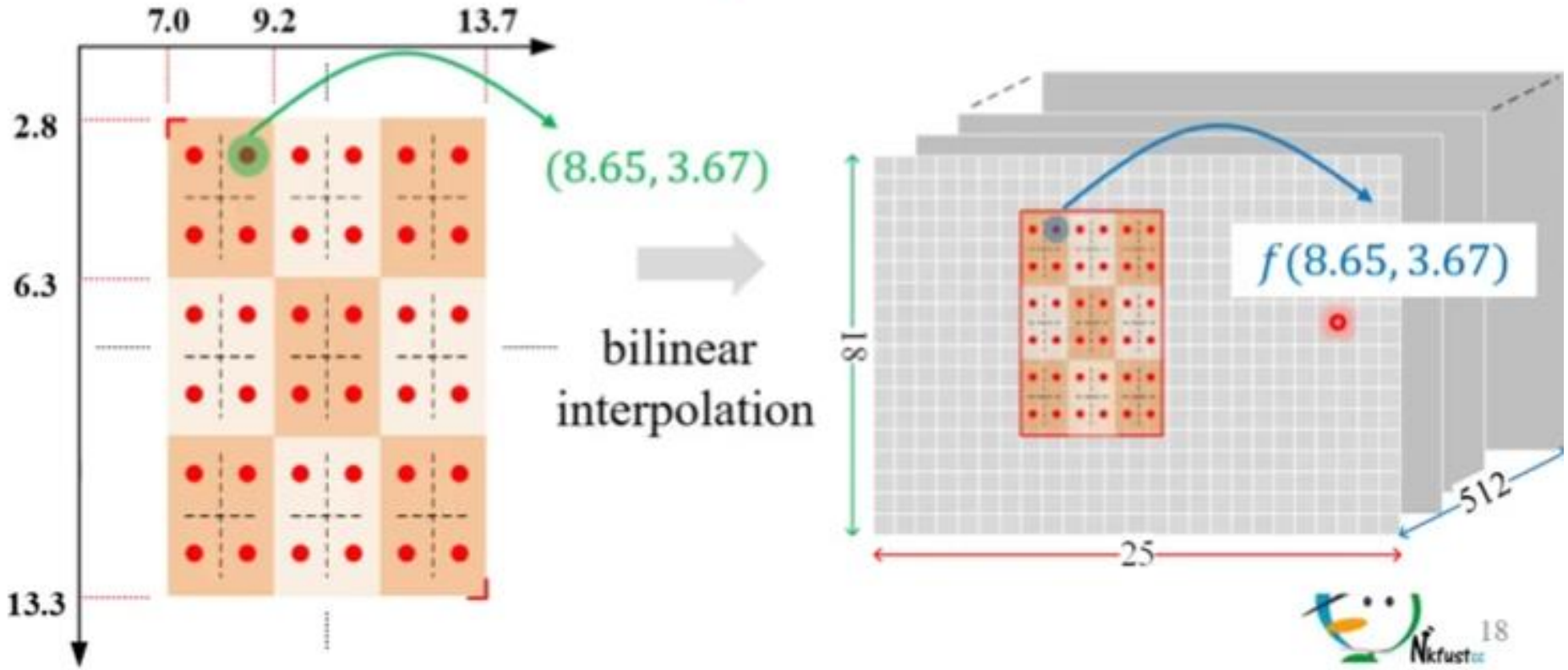
staring coordinate

displacement

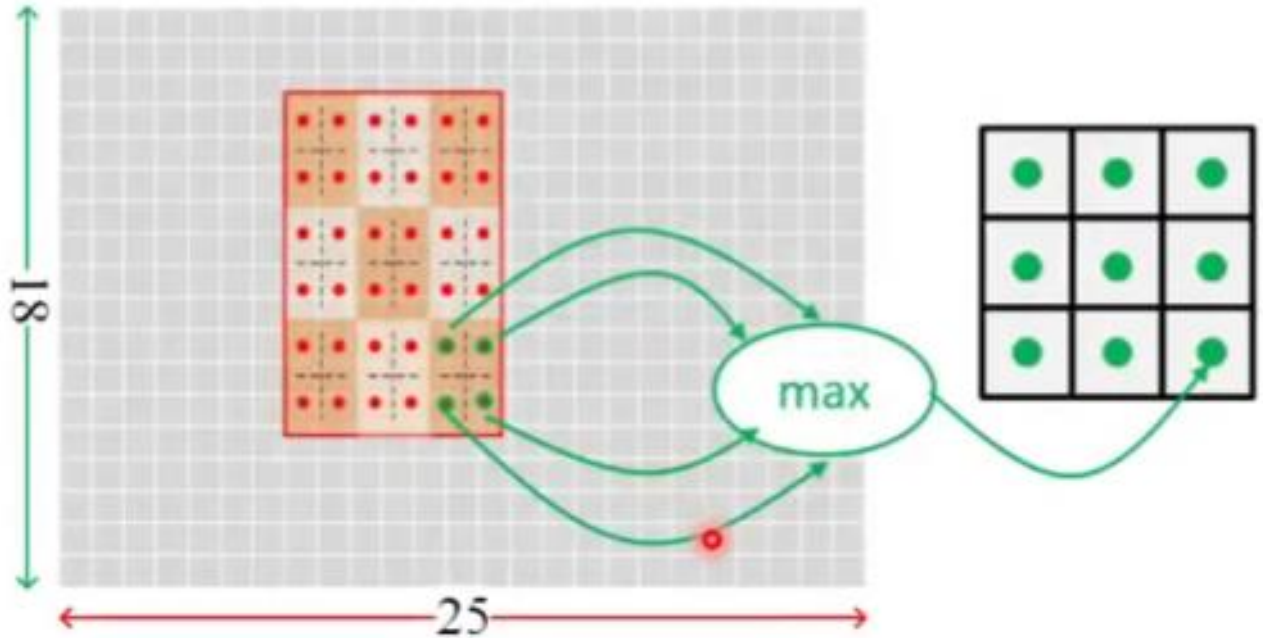
$$x: 7.0 + \left(\frac{2.2}{s = 2}\right) \times (1 + 0.5) = 8.65$$

$$y: 2.8 + \left(\frac{3.5}{s = 2}\right) \times (0 + 0.5) = 3.67$$

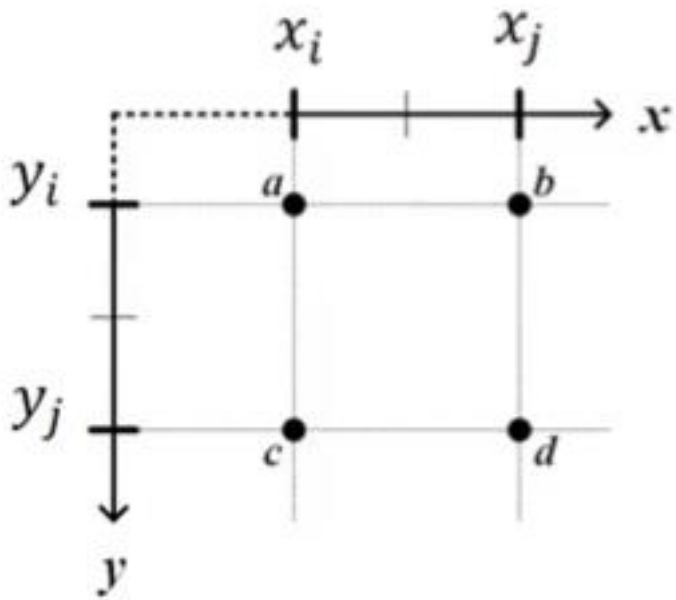
- Align Steps: interpolation
  - interpolate the feature value of every sampling point by bilinear interpolation.



- Align Steps: max pooling
  - take the maximum of feature values of sampling points in each grid
  - aggregate the results in all channels



- Bilinear Interpolation: overview
  - It is a way to interpolate point value on a 2D grid
  - It is performed using linear interpolation twice, respectively, in horizontal and vertical directions



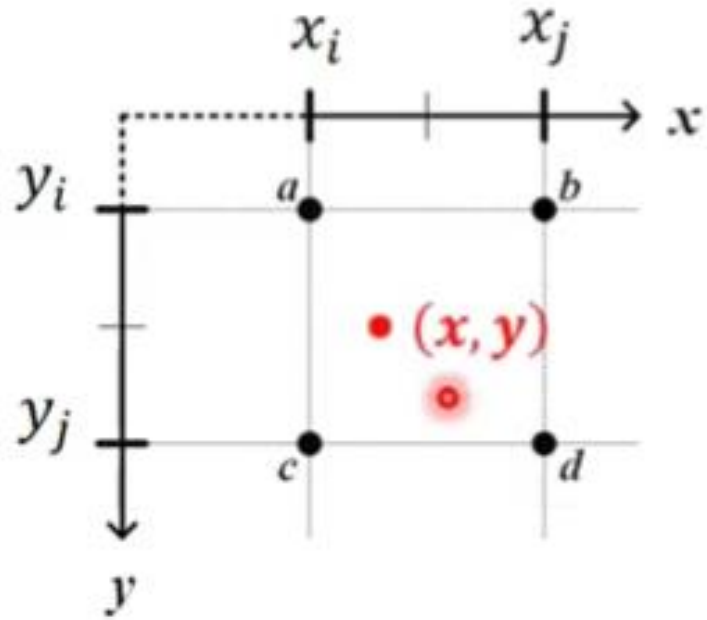
$f(x_i, y_i) = a$

$f(x_j, y_i) = b$

$f(x_i, y_j) = c$

$f(x_j, y_j) = d$

- Bilinear Interpolation: overview
  - It is a way to interpolate point value on a 2D gri
  - It is performed using linear interpolation twice, respectively, in horizontal and vertical directions



$f(x_i, y_i) = a$

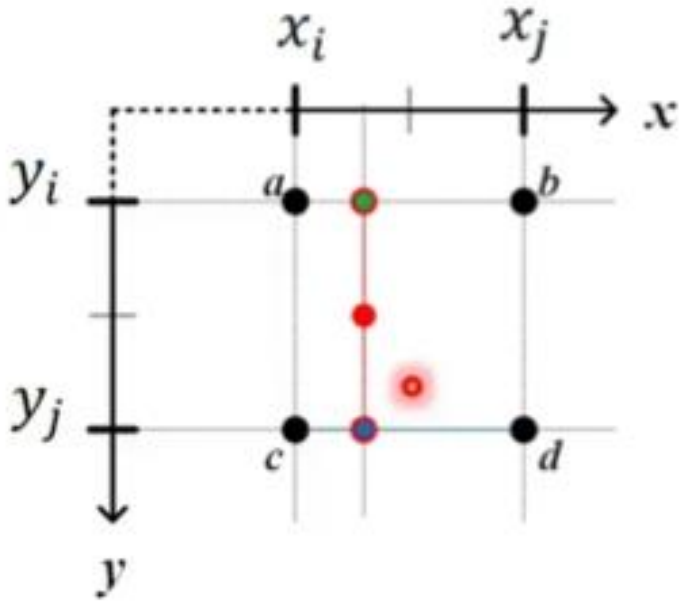
$f(x_j, y_i) = b$

$f(x, y) = ?$

$f(x_i, y_j) = c$

$f(x_j, y_j) = d$

- Bilinear Interpolation: overview
  - It is a way to interpolate point value on a 2D grid
  - It is performed using linear interpolation twice, respectively, in horizontal and vertical directions



$$f(x_i, y_i) = a \rightarrow f(x, y_i) \leftarrow f(x_j, y_i) = b$$

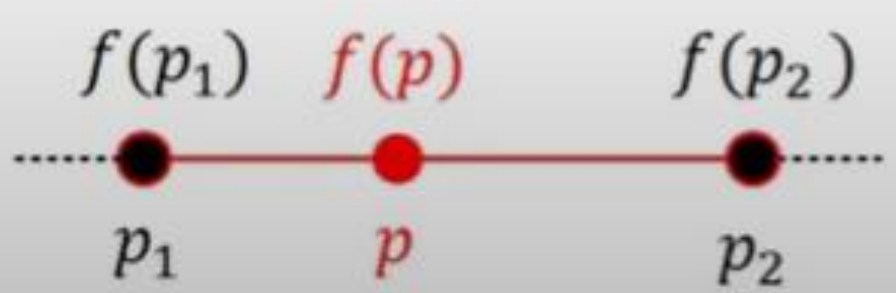
$$\downarrow$$

$$f(x, y) = ?$$

$$\uparrow$$

$$f(x_i, y_j) = c \rightarrow f(x, y_j) \leftarrow f(x_j, y_j) = d$$

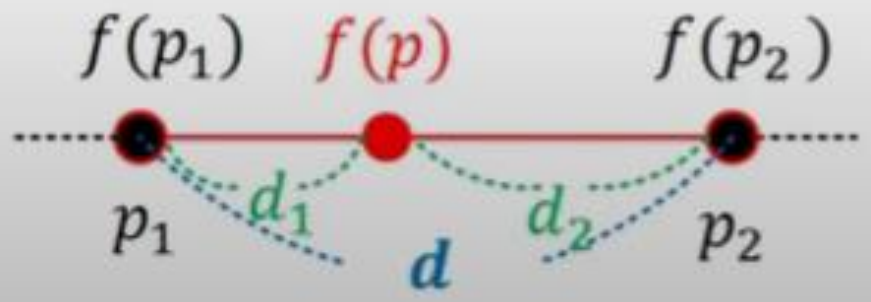
- Bilinear Interpolation: overview
  - Linear interpolation: interpolate the value $f(p)$ of a point p between p_1 and p_2
    - $f(p)$ is the weighted sum of $f(p_1)$ and $f(p_2)$
    - w_1 and w_2 are inversely proportional to d_1 and d_2



$$f(p) = w_1 \times f(p_1) + w_2 \times f(p_2)$$

- $w_1 + w_2 = 1.0$

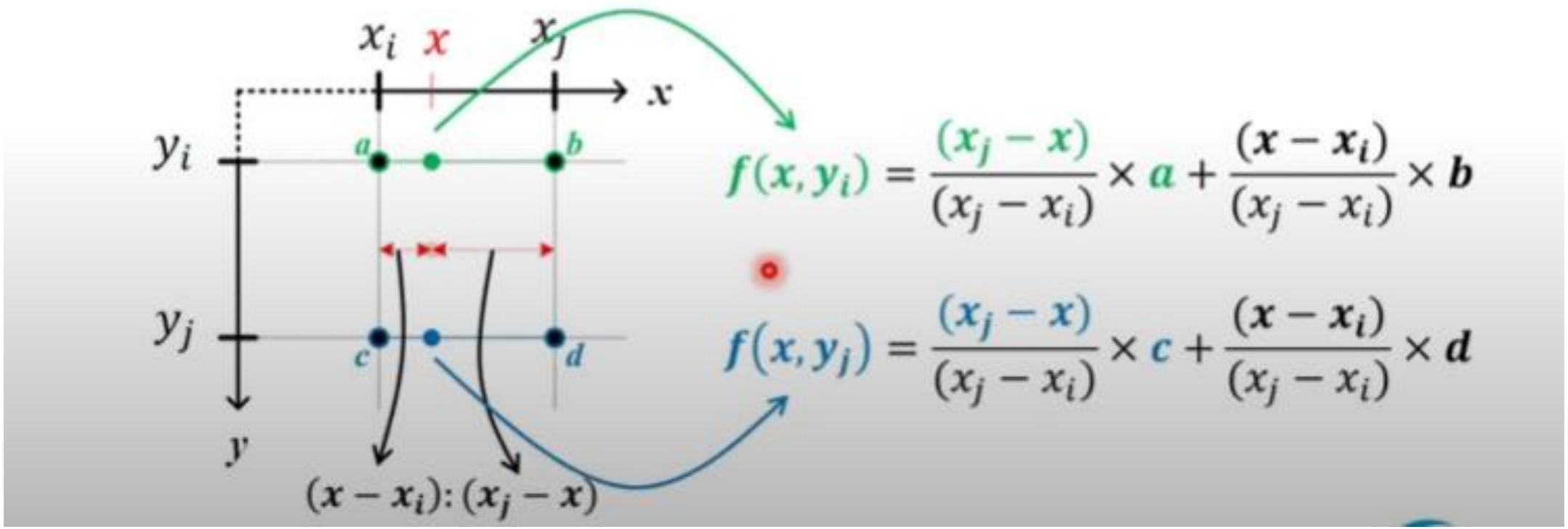
- Bilinear Interpolation: overview
  - Linear interpolation: interpolate the value $f(p)$ of a point p between p_1 and p_2
    - $f(p)$ is the weighted sum of $f(p_1)$ and $f(p_2)$
    - w_1 and w_2 are inversely proportional to d_1 and d_2



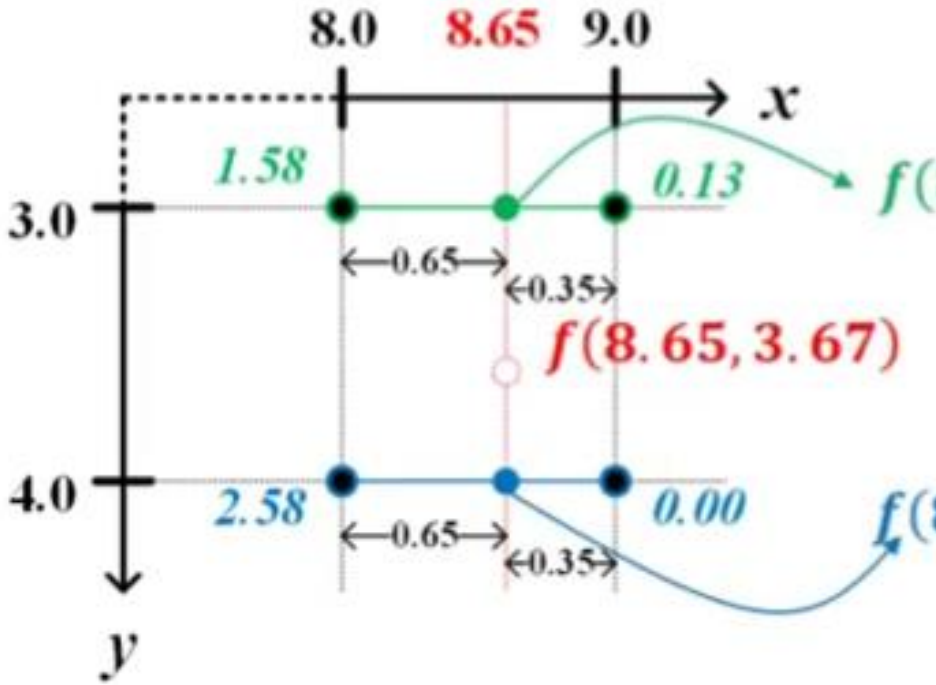
$$f(p) = w_1 \times f(p_1) + w_2 \times f(p_2)$$

- $w_1 + w_2 = 1.0$
- $w_1 \propto \frac{1}{d_1}$
- $w_2 \propto \frac{1}{d_2}$

- Bilinear Interpolation
  - horizontal interpolation: apply linear interpolation in horizontal direction



- Bilinear Interpolation:
  - horizontal interpolation: apply linear interpolation in horizontal direction



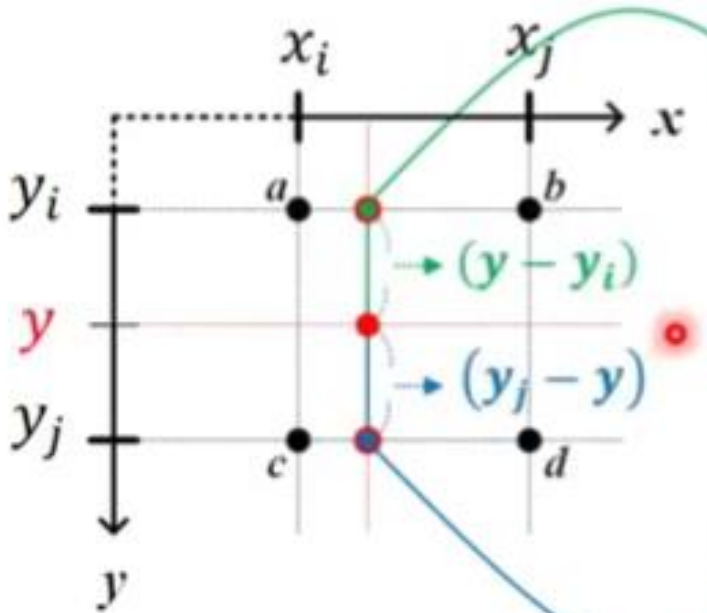
$$f(8.65, 3.0) = \frac{0.35}{1.0} \times 1.58 + \frac{0.65}{1.0} \times 0.13$$

$$= 0.64$$

$$f(8.65, 4.0) = \frac{0.35}{1.0} \times 2.58 + \frac{0.65}{1.0} \times 0.00$$

$$= 0.90$$

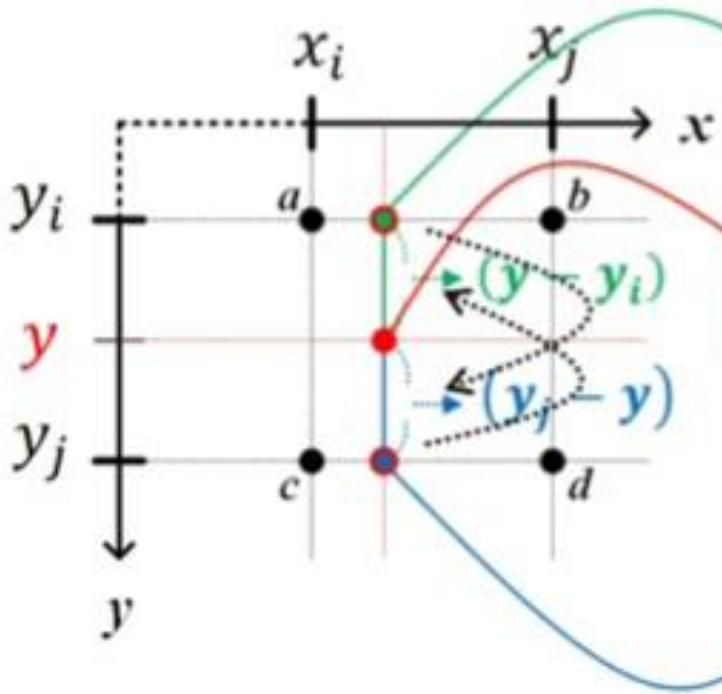
- Bilinear Interpolation
  - vertical interpolation: apply linear interpolation in vertical direction



$$f(x, y_i) = \frac{(x_j - x)}{(x_j - x_i)} \times a + \frac{(x - x_i)}{(x_j - x_i)} \times b$$

$$f(x, y_j) = \frac{(x_j - x)}{(x_j - x_i)} \times c + \frac{(x - x_i)}{(x_j - x_i)} \times d$$

- Bilinear Interpolation
  - vertical interpolation: apply linear interpolation in vertical direction

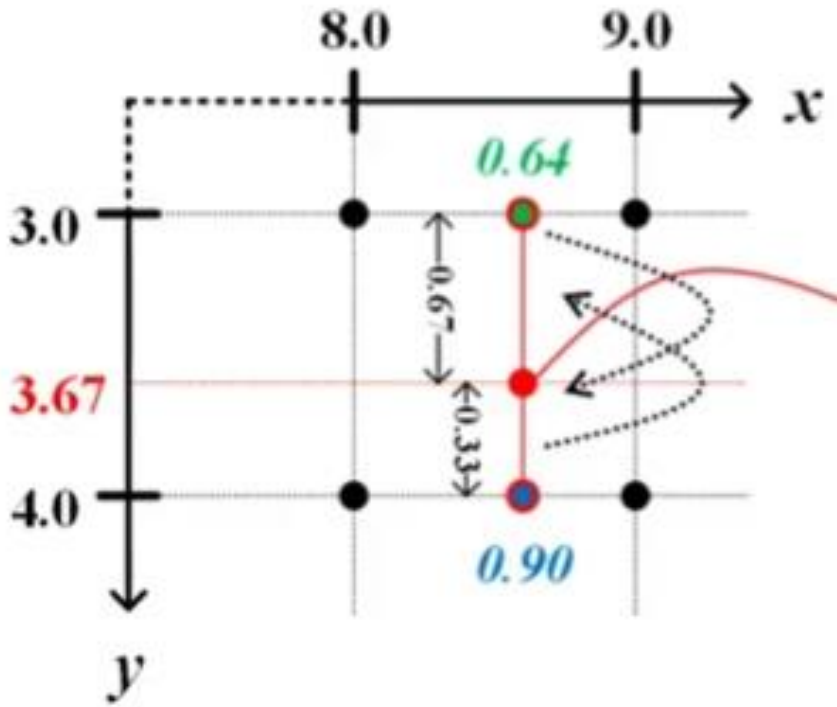


$$f(x, y_i) = \frac{(x_j - x)}{(x_j - x_i)} \times a + \frac{(x - x_i)}{(x_j - x_i)} \times b$$

$$f(x, y) = \frac{(y_j - y)}{(y_j - y_i)} \times f(x, y_i) + \frac{(y - y_i)}{(y_j - y_i)} \times f(x, y_j)$$

$$f(x, y_j) = \frac{(x_j - x)}{(x_j - x_i)} \times c + \frac{(x - x_i)}{(x_j - x_i)} \times d$$

- Bilinear Interpolation
  - vertical interpolation: apply linear interpolation in vertical direction



$$f(8.65, 3.0) = 0.64$$

$$f(8.65, 3.67) = \frac{0.33}{1.0} \times 0.64 + \frac{0.67}{1.0} \times 0.90$$

$$f(8.65, 4.0) = 0.90$$

Resources

- https://www.youtube.com/watch?v=GXYfQsj8RU0&list=PLDiQEXmd_lvcd_ft-qtK5XHoow0xW48Gv&index=4